PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DUOC UC

GUÍA DE CONCEPTOS Y CÓDIGO

Sistema de Gestión y Venta de Gas

"Distribuidora de Gas El Volcán"

Asignatura	DSY1104 — Desarrollo Fullstack II
Integrantes	Nicolás Arancibia Pablo Gómez Benjamín González
Profesor(a)	Hernán Saavedra
Versión	2.1

CHILLÁN · REGIÓN DE ÑUBLE · CHILE

Guía de conceptos del código — Distribuidora de Gas El Volcán (React)

Este documento **traduce y explica** cada cosa que aparece en el código del proyecto, en palabras simples y con **ejemplos tomados de tus propios archivos**. La idea es que puedas leer cualquier línea y entender qué hace, y que te sirva para la **presentación**.

> Cómo usarlo: cuando en el código veas algo que no entiendas, busca la palabra acá con `Ctrl + F`.

Índice

1. JavaScript que aparece en el proyecto
2. React
3. React Router (las rutas)
4. React Bootstrap (los componentes visuales)
5. localStorage (los datos guardados en el navegador)
6. Pruebas unitarias (Vitest)
7. Herramientas (Vite, npm, lint)
8. Cómo se relaciona con tu proyecto
9. Glosario rápido A–Z

---

1. JavaScript que aparece en el proyecto

`const` y `let`

Formas de **crear variables** (una "caja" donde guardas un valor).

- `const` → valor que **no se reasigna** ("constante"). Es lo que más se usa.
- `let` → valor que **sí puede cambiar** después.

```
const nombre = 'El Volcán'   // no cambia
let contador = 0             // puede cambiar
contador = contador + 1
```

En el proyecto casi todo es `const`. Aunque el contenido cambie (por ejemplo un arreglo), la *variable* no se reasigna: por eso es `const`.

> ⚠️ `const` no significa "inmutable". Un arreglo `const` **sí** puede cambiar de contenido; lo que no puede es apuntar a otro arreglo.

Función normal vs. función flecha (arrow)

Una **función** es un bloque de código que hace algo.

```
// Función normal
function sumar(a, b) {
  return a + b
}

// Función flecha (arrow) — la que más se usa en React
const sumar = (a, b) => a + b
```

- Si el cuerpo va entre `{ }`, necesita `return`.
- Si es de **una sola línea**, el `return` es **implícito** (se sobreentiende).
- Ejemplo real en `utils/carrito.js`:

```
export const contarItems = (carrito) =>
  carrito.reduce((suma, item) => suma + (Number(item.cantidad) || 0), 0)
```

Template literals (plantillas de texto)

Usan **backticks** `` ` ` `` y permiten **meter variables** dentro con `${ }`.

```
const id = 'p3'
const url = `/producto/${id}`   // esto vale "/producto/p3"
```

Sirven para armar textos o rutas sin concatenar con `+`. En `Producto.jsx`: `to={`/producto/${props.id}`}`.

Destructuring (desestructuración)

Forma corta de **sacar valores** de un objeto o arreglo.

```
const producto = { id: 'p1', nombre: 'Cilindro', stock: 5 }

// Sin destructuring
const nombre = producto.nombre
const stock = producto.stock

// Con destructuring
const { nombre, stock } = producto
```

En `DetalleProducto.jsx`: `const { id } = useParams()` (saca `id` de lo que devuelve `useParams`). También se usa en los imports: `import { Card, Button } from 'react-bootstrap'`.

Spread `...` (propagación) y rest

El `...` "esparce" el contenido de un objeto o arreglo.

```
const item = { id: 'p1', cantidad: 1 }
const actualizado = { ...item, cantidad: 2 }   // copia y cambia cantidad → { id:'p1', cantidad:2 }

const carrito = [a, b]
const nuevo = [...carrito, c]                  // [a, b, c]
```

Importante: se usa para **crear copias** sin modificar el original (React exige no "mutar" el estado). En `utils/carrito.js` se usa en casi todas las funciones.

Métodos de arreglos (los más usados)

Método	Qué hace	Ejemplo
`.map()`	Recorre y **transforma** cada elemento → nuevo arreglo	`carrito.map((item) => ({ ...item }))`
`.filter()`	Recorre y **deja solo** los que cumplen una condición	`productos.filter((p) => p.stock === 0)`
`.find()`	Devuelve **el primer** elemento que cumple (o `undefined`)	`productos.find((p) => p.id === id)`
`.reduce()`	Recorre **acumulando** un solo resultado (ej. una suma)	`carrito.reduce((s, i) => s + i.cantidad, 0)`
`.some()`	`true` si **alguno** cumple	`items.some((i) => i.cantidad > 0)`

Ejemplo real (`DetalleProducto.jsx`): `catalogoActual().find((p) => p.id === id)` busca el producto por id.

Módulos: `export` / `import`

Cada archivo `.js` / `.jsx` es un **módulo** que puede **exportar** cosas y **importar** de otros.

```
// En archivo A:
export function sumar(a, b) { return a + b }   // export nombrado
export default App                             // export por defecto

// En archivo B:
import App from './App.jsx'                    // import por defecto (sin llaves)
import { sumar } from './util'                 // import nombrado (con llaves)
```

- **Default** (`export default`): uno por archivo; se importa **sin llaves**.
- **Nombrado** (`export const ...`): puede haber varios; se importan **con llaves**.

Operador ternario, `&&`, `||`, `??`, `?.`

Formas cortas de escribir condiciones.

```
// Ternario: condición ? siEsVerdadero : siEsFalso
const texto = sesion ? 'Cerrar sesión' : 'Ingresar'

// && : muestra algo solo si la condición es true
{error && <Alert variant="danger">{error}</Alert>}

// || : "si lo de la izquierda es vacío/falso, usa la derecha"
const carrito = leerLista('carrito') || []

// ?? : parecido a ||, pero solo reacciona a null/undefined (no a 0 ni "")
const stock = item.stock ?? 0

// ?. : accede a una propiedad sin romper si el objeto es null
const direccion = sesion?.direccion || ''
```

JSON: `parse` / `stringify`

Los datos de `localStorage` se guardan como **texto**. `JSON` es el formato.

- `JSON.stringify(objeto)` → convierte objeto a **texto** (para guardar).
- `JSON.parse(texto)` → convierte texto a **objeto** (para leer).

```
localStorage.setItem('carrito', JSON.stringify(carrito))   // objeto → texto
const carrito = JSON.parse(localStorage.getItem('carrito')) // texto → objeto
```

`async` / `await` (adelanto)

Sirven para esperar cosas que **tardan** (como pedir datos a un servidor). Todavía **no se usan** en el proyecto porque no hay backend, pero los verás cuando hagan `fetch()`.

```
const respuesta = await fetch('/api/productos')   // "espera" la respuesta
const datos = await respuesta.json()
```

---

2. React

JSX

Es una **mezcla de HTML y JavaScript** que se escribe dentro de React. Fíjate que en vez de `class` se usa `className`, y las etiquetas siempre se cierran.

```
<h1 className="texto-2">Bienvenido</h1>
<img src={logo} alt="Logo" />
```

Dentro de `{ }` puedes poner **JavaScript** (variables, funciones, ternarios).

Componente

Un **componente** es una **función** que devuelve **JSX** (lo que se ve en pantalla). Su nombre va con **Mayúscula inicial**.

```
function Inicio() {
  return <h1>Hola</h1>
}
export default Inicio
```

Hay componentes **de página** (`pages/`) y **reutilizables** (`components/`).

Props

Son los **datos que un componente recibe** desde afuera. Son de **solo lectura** (no se cambian).

```
function Producto(props) {
  return <h3>{props.nombre}</h3>
}

// Se usa pasando los datos:
<Producto nombre="Cilindro 15kg" precio={12000} />
```

En `Navbar.jsx` : `<Navbar totalItems={totalItems} />` → el componente recibe `totalItems` por props.

`children`

Es lo que va **entre** las etiquetas de apertura y cierre de un componente.

```
<Card.Body>Aquí va el children</Card.Body>
```

`Card.Body` recibe ese contenido como `children` .

Fragment `< >` y `< >...</>`

Permite devolver **varios elementos** sin agregar un `<div>` extra al HTML.

```
function Cosa() {
  return (
    <>
      <h1>Título</h1>
      <p>Texto</p>
    </>
  )
}
```

State con `useState`

El **estado** son datos que **cambian** y que, al cambiar, hacen que React **vuelva a dibujar** la pantalla.

```
import { useState } from 'react'

const [carrito, setCarrito] = useState([])
//     ↑ valor    ↑ función para cambiarlo
```

- `carrito` → el valor actual.
- `setCarrito(nuevoValor)` → cambia el valor y **re-renderiza**.
- `useState([])` → el valor **inicial**.

Ejemplo real (`App.jsx`):

```
const [carrito, setCarrito] = useState(() => {
  try { return JSON.parse(localStorage.getItem(CLAVES.carrito)) || [] }
  catch { return [] }
})
```

> Cuando pasas una **función** a `useState`, React la ejecuta **una sola vez** para calcular el valor inicial (así lee `localStorage` sin repetirlo en cada render).

Para actualizar usamos una función `previo => nuevo`:

```
const agregar = (id) => setCarrito((anterior) => agregarItem(anterior, id))
```

Esto es más seguro porque siempre trabaja con el **estado más reciente**.

`useEffect`

Sirve para hacer algo **después** de que el componente se dibuja (efectos secundarios).

```
useEffect(() => {
  localStorage.setItem(CLAVES.carrito, JSON.stringify(carrito))
}, [carrito])
```

- El primer argumento es la **función** a ejecutar.
- El segundo (`[carrito]`) es la **lista de dependencias**: el efecto se ejecuta cuando `carrito` cambie.
- Con `[]` (vacío) se ejecuta **una sola vez** al montar.

`StrictMode`

Es un componente especial de React (en `main.jsx`) que **no se ve** y **no cambia lo que se muestra**. Sirve para que, en **desarrollo**, React te **avise de errores o malas prácticas**.

```
<StrictMode>
  <App />
</StrictMode>
```

> Ojo: en desarrollo `StrictMode` **ejecuta dos veces** ciertas cosas a propósito (para detectar problemas). En **producción no hace nada**. No es un componente tuyo, viene de React.

Eventos: `onClick` , `onChange` , `onSubmit`

Son funciones que se ejecutan cuando el usuario hace algo.

```
<Button onClick={() => props.onAgregar(props.id)}>Agregar</Button>
<Form.Control onChange={(e) => setEmail(e.target.value)} />
<Form onSubmit={handleSubmit} />
```

- En React los eventos se escriben **camelCase** (`onClick` , no `onclick`).
- `e` es el **evento**; `e.target.value` es lo que el usuario escribió.
- `event.preventDefault()` **evita** que el formulario recargue la página (comportamiento por defecto).

Formularios controlados

Un input es "controlado" cuando su valor **vive en el estado de React** (no en el HTML).

```
const [email, setEmail] = useState('')
<Form.Control value={email} onChange={(e) => setEmail(e.target.value)} />
```

- `value={email}` → el input **muestra** el estado.
- `onChange` → **actualiza** el estado en cada tecla. Así React siempre "manda". Se usa en `Login` , `Registro` , `Contacto` , `Inventario` , `NuevoUsuario` .

Renderizado condicional

Mostrar algo **solo si** se cumple una condición.

```
{error && <Alert variant="danger">{error}</Alert>}
{items.length === 0 ? <p>Vacío</p> : <Lista />}
```

Listas y `key`

Para dibujar muchos elementos se usa `.map()`, y **cada uno necesita una** `key` (un identificador único) para que React los distinga.

```
{productos.map((p) => (
  <Producto key={p.id} {...p} />
))}
```

Sin `key`, React se confunde al actualizar la lista.

Lifting state up (subir el estado)

Cuando **varios componentes** necesitan el mismo dato, ese estado se pone **en el padre común** y se pasa hacia abajo por **props**. En el proyecto, el **carrito** vive en `App` justamente por esto.

Estado derivado

No siempre se guarda todo en el estado: a veces se **calcula** a partir de otro dato.

```
const totalItems = contarItems(carrito)   // se calcula, no se guarda aparte
```

Regla de los Hooks

Los hooks (`useState`, `useEffect`, etc.):

1. Solo se llaman en **componentes** o en **otros hooks** (no en funciones comunes).
2. Siempre van **arriba** del componente, **nunca dentro de un** `if` **o un bucle**.

---

3. React Router (las rutas)

Es la librería que permite tener **varias "páginas"** dentro de una sola app (SPA), **sin recargar** el navegador.

`BrowserRouter`

Envuelve toda la app y **habilita la navegación**. Se pone una sola vez (en `App.jsx`).

```
<BrowserRouter>
  <Routes>...</Routes>
</BrowserRouter>
```

`Routes` y `Route`

- `Routes` → el **contenedor** de las rutas.
- `Route` → una **ruta**: `path` es la URL y `element` es el componente que se muestra.

```
<Route path="/productos" element={<Productos onAgregar={agregar} />} />
<Route path="/producto/:id" element={<DetalleProducto onAgregar={agregar} />} />
```

El `:id` es un **parámetro**: una parte **variable** de la URL (ej. `/producto/p3`).

Rutas anidadas y `Outlet`

Una ruta puede **contener** otras. El componente padre dibuja su parte (ej. el menú) y con `<Outlet />` marca **dónde** va la página hija.

```
<Route element={<LayoutPublico totalItems={totalItems} />}>
  <Route path="/" element={<Inicio />} />
  <Route path="/productos" element={<Productos />} />
</Route>
```

```
// LayoutPublico.jsx
<BotonTienda />
<Navbar totalItems={totalItems} />
<Outlet />          {/* ← aquí aparecen las páginas hijas */}
<Footer />
```

Así el `Navbar` y el `Footer` **no se repiten** en cada página.

Ruta índice (`index`)

Es la ruta que se muestra cuando coincide **solo** la URL del padre (sin agregar nada).

```
<Route path="/admin" element={<AdminLayout />}>
  <Route index element={<Dashboard />} />   {/* /admin → Dashboard */}
  <Route path="inventario" element={<Inventario />} />  {/* /admin/inventario */}
</Route>
```

`Link`

Crea un **enlace** que navega **dentro de la SPA** (sin recargar), en vez de un `<a>` normal.

```
<Link to="/productos">Ver productos</Link>
```

`NavLink`

Como `Link`, pero sabe si la ruta está **activa** (útil para pintar el ítem seleccionado en el menú).

```
<NavLink to="/admin/inventario" className={({ isActive }) => (isActive ? 'active' : '')}>
```

`as={Link}` (con React Bootstrap)

Los componentes de Bootstrap crean un `<a>`; con `as={Link}` les decimos que naveguen como React Router.

```
<Button as={Link} to="/ventas" variant="success">Finalizar compra</Button>
```

`useNavigate`

Devuelve una función para **navegar por código** (ej. después de iniciar sesión o guardar).

```
const navigate = useNavigate()
navigate('/admin')   // te lleva al panel
```

`useParams`

Lee los **parámetros de la URL** (los `:algo`).

```
const { id } = useParams()   // en /producto/p3 → id = "p3"
```

`useLocation` + `URLSearchParams`

`useLocation` da la URL actual; `URLSearchParams` lee lo que va **después del `?`** (query string).

```
const location = useLocation()
const categoria = new URLSearchParams(location.search).get('categoria')
// /productos?categoria=Reguladores → categoria = "Reguladores"
```

`MemoryRouter` (solo en pruebas)

Es un `Router` "de mentira" que vive en memoria. Se usa en los **tests** para poder renderizar componentes que usan `Link` sin abrir un navegador.

```
render(
  <MemoryRouter>
    <Producto {...producto} onAgregar={onAgregar} />
  </MemoryRouter>,
)
```

---

4. React Bootstrap (los componentes visuales)

Son componentes de interfaz ya hechos y **responsivos** (se adaptan a celular/tablet/PC). Ejemplos usados:

Componente	Para qué
`Container`	Centra y da márgenes al contenido
`Row` / `Col`	Grilla responsiva (columnas)
`Card` / `Card.Body` / `Card.Title`	Tarjetas
`Button`	Botones
`Form`, `Form.Control`, `Form.Label`, `Form.Group`	Formularios
`Alert`	Mensajes de error / información
`Badge`	Etiquetas (ej. contador del carrito)
`ListGroup`	Listas
`Table`	Tablas (inventario, usuarios)
`Modal`	Ventanas emergentes
`Navbar`, `Nav`	Barra de navegación

Responsividad con `Col`: `<Col xs={12} md={6} lg={4}>` → 12 columnas (ancho completo) en celular, 6 en tablet, 4 en PC. Bootstrap usa una grilla de **12 columnas**.

---

5. localStorage (los datos guardados en el navegador)

Es un "cajón" del navegador donde se guarda **texto** que **permanece** aunque cierres la pestaña (hasta que se borre). Se usa mientras **no hay backend**.

Operación	Qué hace
`localStorage.setItem(clave, valor)`	Guarda (el valor debe ser texto)
`localStorage.getItem(clave)`	Lee (devuelve texto o `null`)
`localStorage.removeItem(clave)`	Borra

Como solo guarda **texto**, se combina con `JSON`:

```
localStorage.setItem('carrito', JSON.stringify(carrito))   // guardar
const carrito = JSON.parse(localStorage.getItem('carrito')) // leer
```

En el proyecto las **claves** están centralizadas en `utils/validaciones.js` (`CLAVES`): `carrito`, `usuarios`, `sesion`, `productos`, `edicionesProductos`, `ordenes`, `solicitudes`.

Se leen con funciones **seguras** (`leerLista`, `leerObjeto`) que **no rompen** si el texto está corrupto.

---

6. Pruebas unitarias (Vitest)

Una **prueba unitaria** es código que **revisa automáticamente** que otra parte del código funcione. Si algo se rompe más adelante, `npm test` te avisa.

`describe` / `test` / `expect`

```
describe('carrito (lógica pura)', () => {     // agrupa pruebas de un tema
  test('suma las cantidades', () => {         // una prueba puntual
    expect(contarItems([{ cantidad: 2 }])).toBe(2)  // lo que esperas
  })
})
```

- `describe` → agrupa.
- `test` → un caso (describe **qué** debería pasar).
- `expect(valorReal)` → el resultado; seguido de un **matcher**.
- El `matcher` (`.toBe`, `.toEqual`…) expresa **lo esperado**.

Matchers comunes

Matcher	Significa
`.toBe(x)`	es **exactamente** `x`
`.toEqual(x)`	es **igual en contenido** (objetos/arreglos)
`.toBe(true)` / `.toBe(false)`	es verdadero / falso
`.toBeTruthy()` / `.toBeFalsy()`	existe / no existe (verdadero/falso genérico)
`.toBeNull()`	es `null`
`.toBeInTheDocument()`	el elemento está en el DOM (viene de jest-dom)
`.toHaveTextContent('x')`	el elemento **contiene** ese texto
`.toHaveAttribute('href', '/')`	tiene ese atributo con ese valor
`.toHaveValue('hernan')`	un input **tiene** ese valor escrito
`.toHaveBeenCalledWith(x)`	el mock fue llamado con `x`

Mocks / spies (`vi.fn()`)

Un **mock** (o *spy*) es una **función falsa** que "espía" si la llamaron y con qué argumentos. Sirve para probar **sin depender** del componente real.

```
import { vi } from 'vitest'

const onAgregar = vi.fn()                    // función falsa (Vitest)

// ... se renderiza y se hace click ...

expect(onAgregar).toHaveBeenCalledWith('p1')   // ¿la llamaron con 'p1'?
expect(onAgregar).toHaveBeenCalledTimes(1)     // ¿una vez?
```

`beforeEach`

Se ejecuta **antes de cada** prueba, para dejar un estado conocido.

```
beforeEach(() => {
  localStorage.clear()
})
```

Testing Library: `render`, `screen`, `fireEvent` y `user-event`

- `render(<Componente />)` → "dibuja" el componente para probarlo.
- `screen` → busca elementos en lo que se dibujó.
- `fireEvent.click(...)` → **simula** un clic del usuario.
- `userEvent` (`user.type(campo, 'texto')`, `user.click(...)`) → simula la **interacción real** (tecleo, clic).

```
render(<Producto {...producto} onAgregar={onAgregar} />)
fireEvent.click(screen.getByRole('button', { name: 'Agregar' }))
```

```
const user = userEvent.setup()
await user.type(screen.getByLabelText('Correo electrónico'), 'hernan')
```

Consultas (queries)

Función	Busca por...
`screen.getByText('Hola')`	texto visible
`screen.getByRole('button', { name: 'Agregar' })`	su rol y nombre accesible
`screen.getByLabelText('Correo')`	la etiqueta de un input

Función	Busca por...
`screen.queryByText('x')`	igual, pero devuelve `null` si no está (para comprobar AUSENCIA)

`setupTests.js`

Archivo que se ejecuta **antes de todas** las pruebas (se declara en `vite.config.js` → `setupFiles: './src/setupTests.js'`). Carga `@testing-library/jest-dom`, que agrega los matchers de DOM (`toBeInTheDocument`, `toHaveValue`, etc.).

---

7. Herramientas (Vite, npm, lint)

`main.jsx` y `createRoot`

Es el **punto de entrada**: toma el `<div id="root">` del `index.html` y mete la app dentro.

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Importar CSS

```
import 'bootstrap/dist/css/bootstrap.min.css'   // CSS de Bootstrap
import './estilos.css'                           // tus estilos
```

Importar un CSS hace que sus estilos se apliquen a toda la app.

Scripts de `npm` (en `package.json`)

Comando	Qué hace
`npm run dev`	Levanta el servidor de desarrollo (http://localhost:5173)
`npm run build`	Genera la carpeta `dist/` para producción
`npm run lint`	Revisa el estilo/errores del código (oxlint)
`npm test`	Ejecuta las 10 pruebas con Vitest (una sola vez)
`npm run test:watch`	Pruebas en modo vigilancia (se re-ejecutan al guardar)
`npm run test:coverage`	Ejecuta las pruebas y genera el reporte de cobertura en `coverage/`

`.env` e `import.meta.env` (adelanto)

Cuando exista backend, la URL de la API se guardará en un archivo `.env` y se leerá así:

```
const api = import.meta.env.VITE_API_URL
```

Vite "hornea" estas variables en el `build`.

---

8. Cómo se relaciona con tu proyecto

Acá está lo más importante: **dónde vive cada concepto** y **cómo se conectan** las piezas entre sí.

8.1 Mapa: concepto → dónde se usa en TU proyecto

Concepto	Archivo(s) donde aparece	Para qué sirve ahí
`const` / funciones flecha	**Todos**	Declarar variables y funciones.
`export` / `import`	**Todos**	Conectar archivos (App importa páginas, páginas importan utils, etc.).
Props	`App.jsx` → `Navbar`, `Producto`, `Carrito`, `Ventas`, `LayoutPublico`	Pasar el carrito, callbacks y `totalItems` **hacia los hijos**.
`useState`	`App.jsx`, `Login`, `Registro`, `Contacto`, `Inventario`, `NuevoUsuario`, `ListaUsuarios`, `Ventas`, `Nosotros`	Guardar el carrito, los formularios, los filtros y el blog expandido.
`useEffect`	`App.jsx`	Persistir el carrito en `localStorage` cuando cambia.
`StrictMode`	`src/main.jsx`	Avisos de errores/malas prácticas en desarrollo.
Fragmento `< >`	`LayoutPublico.jsx`, `AdminLayout.jsx`, `Nosotros.jsx`	Devolver varios elementos sin un `<div>` extra.
Listas + `key`	`Productos`, `Consejos`, `Nosotros`, `Inventario`, `ListaUsuarios`, `Dashboard`	Dibujar tarjetas, filas y tablas con `.map()`.
Formularios controlados	`Login`, `Registro`, `Contacto`, `Inventario`, `NuevoUsuario`	`value` + `onChange` conectados al estado.
Renderizado condicional	`Carrito`, `DetalleProducto`, `Login`, `Nosotros`, `Inventario`	Mostrar `Alert` de error/vacío, "producto no encontrado", etc.

Concepto	Archivo(s) donde aparece	Para qué sirve ahí
`BrowserRouter` / `Routes` / `Route`	`App.jsx`	Definir toda la navegación de la SPA.
Rutas anidadas + `Outlet`	`App.jsx` + `LayoutPublico.jsx` + `AdminLayout.jsx`	Compartir Navbar/Footer o el menú admin sin repetir código.
`Link`	`Navbar`, `Inicio`, `Producto`, `Carrito`, `DetalleProducto`, `BotonTienda`	Navegar entre páginas sin recargar.
`as={Link}`	`Inicio`, `Productos`, `Carrito`, `DetalleProducto`, `Producto`, `Nosotros`, `Consejos`	Botones de Bootstrap que navegan como Router.
`NavLink`	`AdminLayout.jsx`	Marcar el ítem activo del menú lateral.
`useParams`	`DetalleProducto.jsx`	Leer el `:id` de `/producto/:id`.
`useLocation` + `URLSearchParams`	`Productos.jsx`	Leer `?categoria=` para filtrar el catálogo.
`useNavigate`	`Login`, `Registro`, `Ventas`, `AdminLayout`	Redirigir por código (tras login, registro o cerrar sesión).
`MemoryRouter`	`Producto.test.jsx`, `Navbar.test.jsx`, `Login.test.jsx`	Dar un Router falso en las pruebas.
`localStorage`	`App.jsx`, `utils/validaciones.js`, `utils/productos.js`, `Login`, `Registro`, `Contacto`, `Ventas`, `Inventario`, `ListaUsuarios`, `NuevoUsuario`	Guardar carrito, usuarios, sesión, productos, órdenes y solicitudes.
`JSON.parse` / `stringify`	`App.jsx`, `utils/validaciones.js`	Convertir objetos ↔ texto para guardar/leer.
`map` / `filter` / `find` / `reduce`	`utils/carrito.js`, `utils/productos.js`, `Productos`, `Inventario`, `Dashboard`, `ListaUsuarios`	Transformar, filtrar, buscar y sumar.
**Ternario / `&&` / ``		`/` ?? `/` ?.`**
Mocks / spies	`Producto.test.jsx` (`vi.fn()`)	Espiar si se llamó `onAgregar` con el id correcto.
`render` / `screen` / `fireEvent`	`Producto.test.jsx`, `Navbar.test.jsx`, `Nosotros.test.jsx`	Dibujar componentes y simular clics en las pruebas.
`user-event` (`user.type`)	`Login.test.jsx`	Simular que un usuario escribe en un formulario.

8.2 Recorrido: qué hace cada archivo y qué conceptos aplica

Archivo	Rol en el proyecto	Conceptos que aplica
`src/main.jsx`	Punto de entrada: monta la app y carga los CSS.	`createRoot`, `StrictMode`, import de CSS.
`src/App.jsx`	**El cerebro**: rutas + estado global del carrito.	`BrowserRouter`, `Routes` / `Route`, rutas anidadas, `useState`, `useEffect`, props.
`components/LayoutPublico.jsx`	Layout público (menú + contenido + pie).	Props, `Outlet`, Fragmento.
`components/Navbar.jsx`	Menú superior (en todas las páginas públicas).	Props (`totalItems`), `Link`, `Badge`.
`components/Producto.jsx`	Tarjeta de producto reutilizable.	Props, callback hijo→padre (`onAgregar`), `Link`.
`components/Footer.jsx` / `BotonTienda.jsx`	Pie de página y botón flotante a Tiendas.	Presentación; `Link`.
`pages/Inicio.jsx`	Portada.	Presentación, `Link` / `as={Link}`.
`pages/Productos.jsx`	Catálogo + filtro por categoría.	`useLocation`, `URLSearchParams`, `.map()`, `.filter()`, props.
`pages/DetalleProducto.jsx`	Ficha de un producto.	`useParams`, `.find()`, renderizado condicional, `as={Link}`.
`pages/Carrito.jsx`	Carrito con cantidades y total.	Props, `.map()`, renderizado condicional, uso de `utils/carrito.js`.
`pages/Ventas.jsx`	Checkout: dirección, pago y orden.	`useState`, `useNavigate`, `localStorage`, `calcularTotal`.
`pages/Login.jsx` / `Registro.jsx`	Acceso y creación de cuenta.	Formularios controlados, validaciones, `useNavigate`, `localStorage`.
`pages/Nosotros.jsx`	Historia con blogs expandibles.	`useState` (estado local), `.map()`.
`pages/Consejos.jsx` / `Tiendas.jsx`	Información de seguridad y sucursales.	Presentación, `.map()`.
`pages/Contacto.jsx`	Formulario de contacto.	Formulario controlado, validaciones, `localStorage`.
`pages/admin/AdminLayout.jsx`	Layout del panel con menú lateral.	`NavLink`, `Outlet`, `useNavigate`, `localStorage` (cerrar sesión).
`pages/admin/Dashboard.jsx`	Indicadores y stock crítico.	`.filter()`, `.map()`, `formatearPrecio`.
`pages/admin/Inventario.jsx`	CRUD de productos.	`useState`, validaciones, `localStorage`, `.map()`, condicionales, `Modal`.

Archivo	Rol en el proyecto	Conceptos que aplica
`pages/admin/ListaUsuarios.jsx`	Listar y filtrar usuarios.	`useState` (filtro), `.map()`, `.filter()`, `localStorage`.
`pages/admin/NuevoUsuario.jsx`	Crear usuarios.	Formulario controlado, validaciones, selectores Región→Comuna.
`utils/validaciones.js`	Validaciones y datos compartidos (funciones puras).	Regex, funciones flecha, `export`, `JSON`, helpers de `localStorage`.
`utils/carrito.js`	Operaciones del carrito (puras).	`.map()`, `.filter()`, `.reduce()`, spread `...`, no mutar.
`utils/productos.js`	Catálogo real (ediciones + creados).	`.map()`, `.find()`, spread, `localStorage`.
`datos/productos.js`	Los 14 productos base.	Arreglo de datos, `export`.

8.3 El flujo completo: "agregar un producto y comprarlo"

Así se conecta **todo el proyecto** de principio a fin (ideal para explicar en la presentación):

1. **El clic** → en `components/Producto.jsx`, el botón ejecuta `onClick={() => props.onAgregar(props.id)}`.
2. **El aviso al padre** → ese `onAgregar` es la función `agregar` que `App.jsx` le pasó **por props** (a través de `Productos`). Es el patrón **hijo → padre**.
3. **Cambia el estado** → en `App.jsx`, `agregar` llama a `setCarrito((anterior) => agregarItem(anterior, id))`. La función `agregarItem` (de `utils/carrito.js`) **devuelve un carrito nuevo** con `.map()` y spread, **sin modificar el original**.
4. **React repinta** → al cambiar el **estado**, React **re-renderiza** los componentes que lo usan.
5. **Se guarda** → el `useEffect` de `App.jsx` hace `localStorage.setItem('carrito', JSON.stringify(carrito))`, así **no se pierde al recargar**.
6. **Se refleja en el menú** → `const totalItems = contarItems(carrito)` se recalcula y baja **por props** al `Navbar`, que lo muestra en el `Badge` del carrito.
7. **Se ve el carrito** → en `pages/Carrito.jsx`, `obtenerItemsCompletos` **une** el carrito con el catálogo (`.find()`) y `calcularTotal` **suma** (`.reduce()`) el total.
8. **Finalizar compra** → el botón `as={Link} to="/ventas"` navega al checkout. En `pages/Ventas.jsx`, con `useState` se piden dirección y método de pago; se **genera la orden** (`ORD-######`) y se guarda en `localStorage.ordenes`; finalmente se **vacía el carrito**.

> En una frase: **`Producto` avisa → `App` cambia el estado → `carrito.js` calcula → `useEffect` guarda → `Navbar` y `Carrito` se actualizan → `Ventas` cierra la compra.**

9. Glosario rápido A–Z

Término	En palabras simples
API	"Mesero" entre el frontend y los datos; se pide y responde en JSON.
Componente	Función que devuelve JSX (una parte de la pantalla).
DOM	El árbol de elementos HTML que el navegador muestra.
Estado (state)	Datos que cambian y que redibujan la pantalla.
Hook	Función especial de React (`useState` , `useEffect` , `useParams` ...).
JSX	"HTML con JavaScript" dentro de React.
localStorage	Cajón de texto del navegador que persiste.
Mock / spy	Función falsa para espiar llamadas en los tests.
Props	Datos que un componente recibe (solo lectura).
Renderizar	Dibujar en pantalla.
Router	Sistema de rutas de la SPA.
SPA	App de una sola página que cambia contenido sin recargar.
Build	Compilar el proyecto para producción (`dist/`).

Documento de apoyo para la asignatura DSY1104 — Desarrollo Full Stack II.